LAPRAK TEKNOLOGI IOT 2

TUGAS PRAKTIKUM 3

Oleh:

NAMA : ERVAN GUSTI WANJAYA

NPM : 25782096

KELAS : TRI – 3D

Dosen:

Dr. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd.,M.Kom.

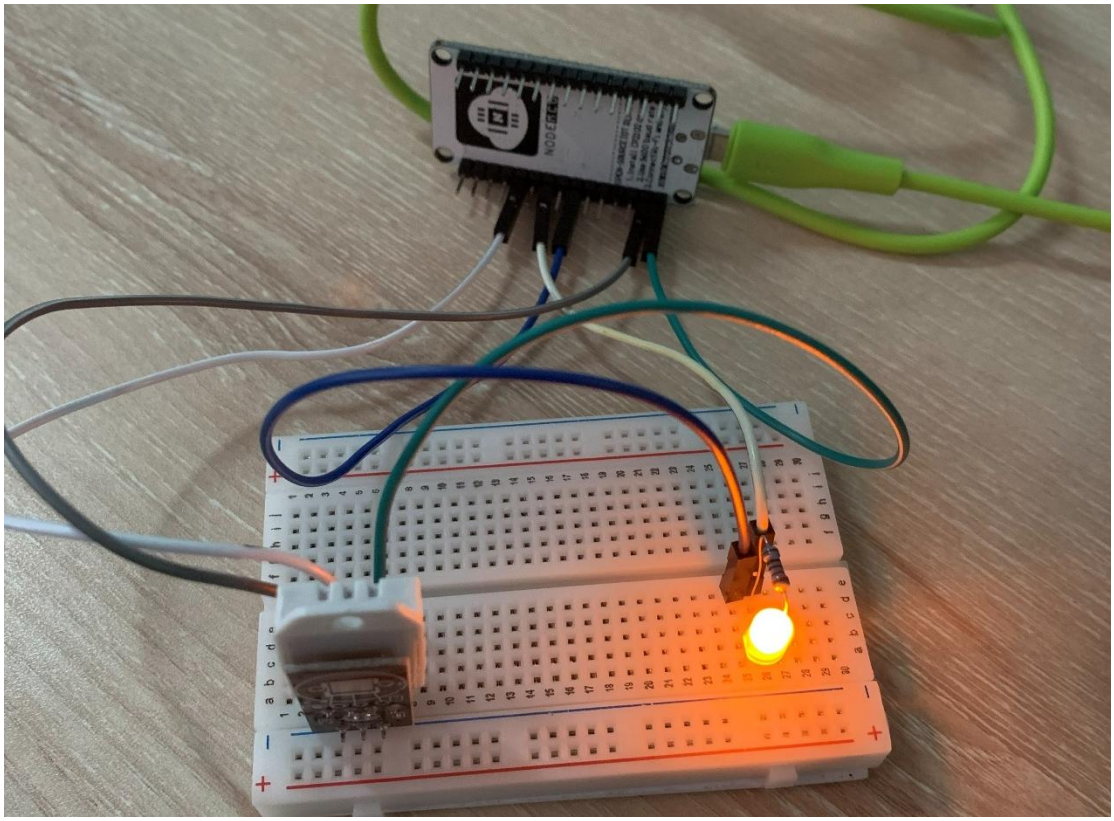
TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

3.5.1 Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)





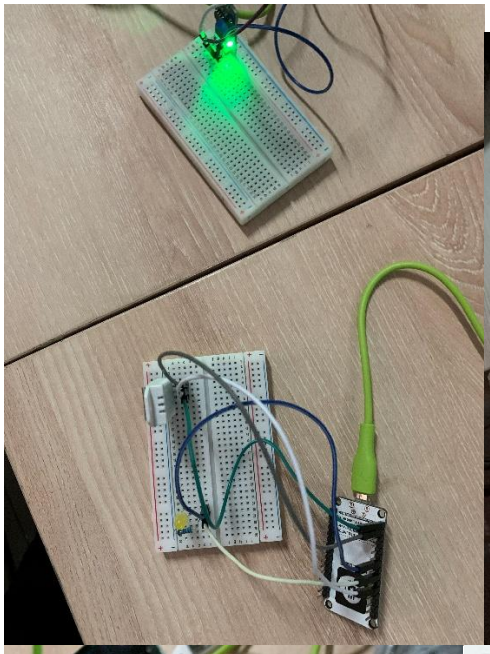
ESP8266 Web Server

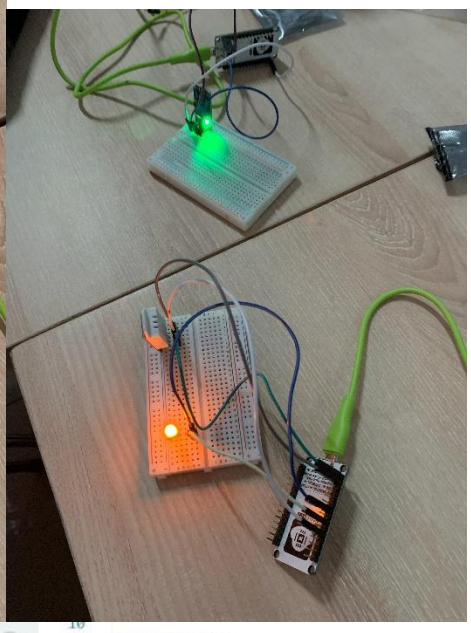
Suhu Saat Ini: 30.60 Celcius

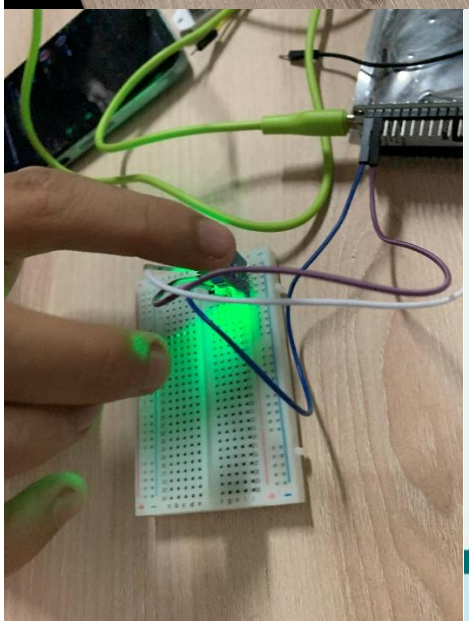
Kendali Relay

ON OFF

3.5.2 Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)







```
11  void setup() {
12    Serial.begin(115200);
13    WiFi.mode(WIFI_STA);
14    WiFi.begin(ssid, password);
15    while (WiFi.status() != WL_CONNECTED) { delay(500
16    Serial.println("\nClient Terhubung ke Wi-Fi!");
17  }
18
19  void loop() {
20    int ldrValue = analogRead(ldrPin);
21
```

Output Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM

```
HTTP Response code: -1
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
```

Building sketch

86°F
Mostly sunny

3.6 Latihan

Latihan 1: Kita menggunakan mode jaringan Station (STA) agar kedua perangkat dapat terhubung ke *router* atau *hotspot* eksternal yang sama sehingga berada dalam satu *subnet* jaringan untuk saling berkomunikasi.

Latihan 2: Modifikasi dilakukan dengan menambahkan pembacaan dht.readHumidity(), menyisipkan placeholder %HUMIDITY% serta %RELAY_BUTTON% pada HTML, lalu memperbarui fungsi handleRoot() untuk menampilkan nilai kelembapan dan tombol *toggle* dinamis berdasarkan status relay.



Latihan 3: *Serial Monitor* pada perangkat *Client* akan mencetak kode respons negatif atau pesan eror koneksi karena fungsi http.GET() gagal menjangkau alamat IP *Server* yang sedang tidak aktif.

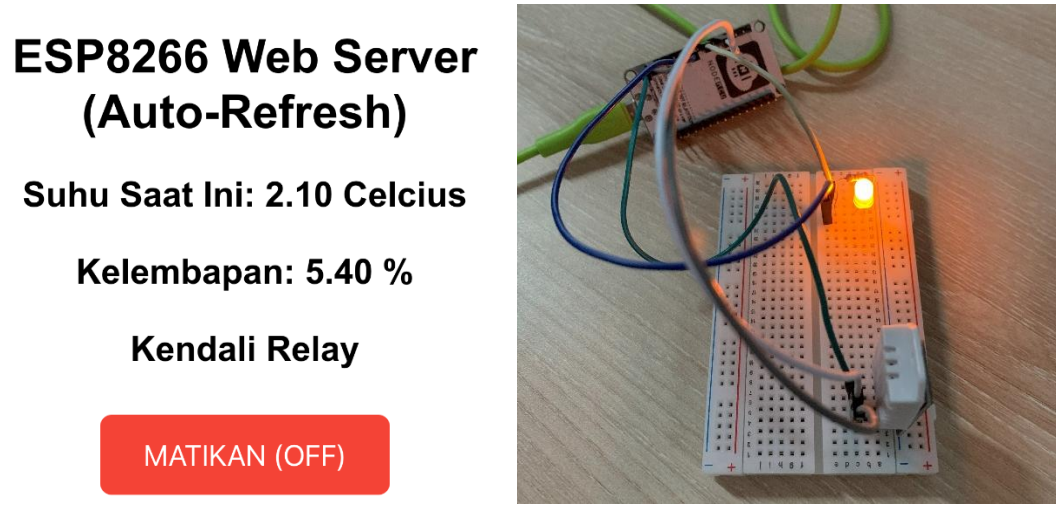
Latihan 4: Blok kondisinya diubah menjadi **if ((WiFi.status() == WL_CONNECTED) && (!isnan(t)) && (t > 35.0))** agar *Client* mengirim permintaan HTTP GET hanya saat sensor DHT membaca suhu di atas 35°C.

```
float t = dht.readTemperature();

// Mengirim permintaan HANYA JIKA suhu melebihi 35 Celcius
if ((WiFi.status() == WL_CONNECTED) && (!isnan(t)) && (t > 35.0)) {
  WiFiClient client;
  HTTPClient http;
  // ... (kode http.GET dilanjutkan di sini)
}
```

3.7 Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur





Berdasarkan hasil pengujian *Web Server* dengan penambahan tag **<meta http-equiv="refresh" content="5">**, ditemukan beberapa kelemahan visual dan interaksi yang signifikan (*flickering issues*):

1. **Efek Kedipan Visual (*Screen Flickering / White Flash*):** Setiap kali interval waktu 5 detik tercapai, *browser* klien dipaksa untuk memuat ulang (*reload*) seluruh halaman web dari awal. Hal ini menyebabkan layar mengalami kedipan warna putih sesaat secara berkala setiap kali siklus pembaruan data terjadi, sehingga menurunkan kenyamanan visual (*user experience*) bagi pengamat.
2. **Pemborosan *Bandwidth* dan Beban Mikrokontroler:** Metode ini bekerja dengan mengirimkan ulang seluruh dokumen HTML, struktur elemen, hingga lembar gaya CSS secara utuh dari memori *Flash* NodeMCU setiap 5 detik. Padahal, data aktual yang mengalami perubahan di lingkungan sebenarnya hanyalah nilai variabel suhu dan kelembapan saja. Hal ini sangat tidak efisien bagi mikrokontroler berdaya rendah.
3. **Potensi Bentrokan Permintaan (*Request Collision / Race Condition*):** Terdapat risiko kegagalan interaksi ketika pengguna mengeklik tombol kendali (seperti tombol *ON* atau *OFF* pada *relay*) di antarmuka *browser* tepat di detik yang bersamaan dengan jalannya proses *auto-refresh* otomatis dari sistem. Hal ini dapat mengakibatkan perintah klik terputus, tidak direspons oleh server, atau membuat halaman web menjadi tersendat (*lag*).

Kesimpulan: Protokol HTTP GET statis dengan *Meta Refresh* tidak ideal untuk diandalkan sebagai sistem *Real-Time Dashboard* jangka panjang. Sistem IoT modern umumnya beralih menggunakan teknik asinkronus berbasis *AJAX/Fetch API* atau protokol *pub/sub* seperti MQTT agar pembaruan data

sensor dapat dikirim secara mandiri tanpa harus memuat ulang seluruh halaman web.